

FINAL TASK DISTRIBUTION

Member 1 — ML Pipeline Lead (Training + DVC + MLflow)

Main Responsibility

Everything related to:

- model training
 - pipeline orchestration
 - experiment tracking
-

Tasks

1. DVC Pipeline (CRITICAL)

- Initialize DVC
- Create `dvc.yaml` with stages:
 - prepare
 - preprocess
 - featurize
 - train

Ensure:

`dvc repro`

- works end-to-end
-

2. Training Pipeline

Folder:

`src/training/`

Implement:

- `train.py`
 - `hpo.py` (Grid / Random / Optuna)
-

3. Model Evaluation

`src/evaluation/`

- accuracy / metrics
 - validation logic
-

4. MLflow (VERY IMPORTANT)

- Start tracking server
- Log:
 - params
 - metrics
 - artifacts
- Run ≥ 3 experiments
- Register best model

Move:

None $\rightarrow$ Staging $\rightarrow$ Production

-
-

5. Model Artifact

- Save model OR use MLflow registry
 - Provide to Ibrahim
-

Files Owned

`dvc.yaml`

`dvc.lock`

`src/training/`

`src/evaluation/`

Member 2 — Data & Preprocessing Lead

Main Responsibility

Everything related to:

- data
- preprocessing
- feature engineering
- validation

Tasks

1. Data Handling

src/data/

- load data
- clean data
- handle missing values

2. Preprocessing Pipeline (MANDATORY)

- Build `sklearn Pipeline`
- Include:
 - imputation
 - scaling
 - encoding
 - (SMOTE / feature selection)

3. Feature Engineering

src/features/

- transformations
 - feature creation
-

4. Config System

configs/params.yaml

- ALL parameters go here
 - no hardcoding
-

5. Data Validation (CI requirement)

- Pandera / Great Expectations
 - schema checks
-

6. Unit Tests

tests/unit/

- at least 3 tests for preprocessing
-

Files Owned

src/data/
src/features/
configs/params.yaml
tests/data/
tests/unit/

Member 3 — Deployment + CI/CD + Monitoring Lead

Main Responsibility

Everything after training:

- API
- CI/CD
- monitoring
- Docker

Tasks

1. Model Serving (FastAPI)

src/serving/

Implement:

- `/health`
 - `/predict`
 - `/predict/batch` (bonus)
-

2. API Validation

- input schema (Pydantic)
 - type checking
-

3. API Tests

tests/integration/

- test `/predict`
-

4. CI/CD Pipeline

.github/workflows/ci.yml

Must include:

1. Lint
 2. Tests ($\geq 70\%$ coverage)
 3. Data validation
 4. Model validation
-

5. Monitoring & Drift Detection

monitoring/

- Evidently reports:
 - baseline
 - drift
 - drift logic ($> 20\%$)
-

6. Prometheus Metrics

- inference count
 - model version
 - feature distributions
-

7. Docker (BONUS)

- Dockerfile
 - docker-compose
-

Files Owned

src/serving/
monitoring/
.github/
Dockerfile
docker-compose.yml
requirements.txt
tests/integration/

SHARED RESPONSIBILITIES (ALL MEMBERS)

Documentation (MANDATORY)

docs/
README.md

Split like this:

Section	Owner
Data & preprocessing	Member 2
Training & MLflow	Member 1
API & deployment	Member 3
Final editing	ALL

INTERFACE AGREEMENTS (VERY IMPORTANT)

Member 2 → Member 1

Output:

data/splits/train.csv
data/splits/test.csv

Must agree on:

- column names
 - target column
-

Member 1 → Member 3

Output:

- `model.pkl` OR MLflow model

Must agree on:

- feature order
 - input schema
-

Member 3 → Monitoring

Uses:

- predictions
 - production data
-

STRICT RULES (TO AVOID CONFLICTS)

Only ONE person edits:

File	Owner
<code>dvc.yaml</code>	Member 1
<code>params.yaml</code>	Member 2
<code>ci.yml</code>	Member 3
<code>requirements.txt</code>	Member 3

NEVER:

- push to `main`
 - work directly on `develop`
 - edit someone else's module
-

ALWAYS:

- use feature branches
- open PR
- review before merge